Monitor Page — Info Document

Page Overview

Purpose

The Monitor page is the real-time observability surface for every execution in progress on the local device. It shows a tab strip with one tab per active execution (manual Deep Dive, manual Deep Sweep, or routine-fired Automated Sweep), and a focused detail pane that renders the pipeline stages, per-repository progress grid, aggregate counters, and a live activity log for whichever tab is currently focused. Terminal executions (completed, failed, cancelled) remain on the page until the user explicitly dismisses them.

Primary User Flows

- **Watch a single run.** The user launches a Deep Dive or Deep Sweep from another page, the new execution is auto-focused on the Monitor tab strip, and the detail pane begins rendering live progress events as they arrive.
- **Switch between concurrent runs.** When two or more executions are active at once (any mix of manual and routine-fired), the user clicks a tab — or presses `Enter` / `Space` on a focused tab — to shift the detail pane to that execution.
- **Toggle verbose logging.** A checkbox in the toolbar controls whether `INFO` -level lines appear in the Live Activity Log. `WARN` and `ERROR` lines always show. The preference is persisted in `localStorage` under the key `resmon.verboseLogging` .
- **Cancel a running execution.** The Cancel button in the `ExecutionHeader` component calls the cooperative-cancel endpoint; the row transitions through the `cancelling` state and finalizes as `cancelled` once the backend flushes partial results.
- **Dismiss a finished run.** After an execution reaches a terminal status, the per-tab `×` button becomes enabled and removes that execution from the in-memory store; the toolbar also exposes a "Clear Page" button that dismisses the focused execution.
- **Jump to the finalized report.** Once terminal, the toolbar exposes a "View Report" button that navigates to `/results?exec=<id>` .

Inputs and Outputs

- **Inputs:**
  - In-memory `ActiveExecution` records maintained by `ExecutionContext` (one per tracked execution id).
  - `GET /api/routines` — used to translate `routine_id` values on routine-fired executions into display names.
  - `GET /api/executions/{id}` — used once per new tab to enrich the `ActiveExecution` with its originating `routine_id`.
  - `GET /api/executions/{id}/progress/events` — the per-second event poll that drives the detail pane.
  - `GET /api/executions/active` — a 3-second safety-net poll that attaches background-initiated runs (e.g. APScheduler-fired routines) and detects dropouts.
- **Outputs:**
  - DOM rendering of the tab strip, toolbar, `ExecutionHeader`, `PipelineStages`, `RepoProgressGrid`, `StatsCounters`, and `LiveActivityLog`.
  - `POST /api/executions/{id}/cancel` when the user cancels.
  - Navigation side effects to `/results?exec=<id>`.
  - `resmon:execution-completed` window events broadcast on terminal transitions (consumed by pages that do not read `ExecutionContext` directly).

Known Constraints

- The Monitor page is purely a view of `ExecutionContext` state; it does not itself open the SSE stream. All live updates arrive via the `/api/executions/{id}/progress/events` poll spawned by `ExecutionContext`.
- The per-tab `×` close button is disabled while the execution's status is `running` or `cancelling`; dismissal is only permitted for `completed`, `failed`, or `cancelled`.
- The page cannot reopen an execution after it has been dismissed. Dismissal deletes the in-memory `ActiveExecution` record and stops its poller; the persisted report remains accessible via Results & Logs.
- OS desktop notifications require `Notification.permission === 'granted'` and the per-type notification-settings predicate to pass; on browsers without the `Notification` API the notification step is skipped silently.

Frontend

Route and Main Component

- **Route:** `/monitor` (registered in `App.tsx`).
- **Main component:** `MonitorPage` (`resmon_scripts/frontend/src/pages/MonitorPage.tsx`).

Child Components and Hooks

- `PageHelp` — inline help panel keyed by `storageKey="monitor"` ; renders two sections ("What this page shows" and "Controls").
- `ExecutionHeader` — status, type label, cancel button.
- `PipelineStages` — ordered stage chips driven by `currentStage` (stage values such as `querying` , `dedup` , `linking` , `reporting` , `summarizing` , `finalizing`).
- `RepoProgressGrid` — per-repository status tiles driven by `repoStatuses` .
- `StatsCounters` — aggregate `resultCount` / `newCount` counters.
- `LiveActivityLog` — renders the event list; filters out `INFO` lines unless `verbose` is `true` .
- Hooks:
  - `useExecution()` — the single source of truth for executions state.
  - `useNavigate()` — used for the "View Report" button.
  - Local `useState` / `useEffect` / `useMemo` for the routines catalogue and the per-execution routine id lookup map.

UI State Model

- **Consumed from `ExecutionContext` via `useExecution()` :**
  - `activeExecutions: Record<number, ActiveExecution>` — keyed by execution id.
  - `executionOrder: number[]` — append-only insertion order; drives tab rendering.
  - `focusedExecutionId: number | null` — id of the currently focused tab.
  - `focusExecution(id)` — sets the focused id.
  - `clearExecution(id?)` — removes an execution from the store (defaults to the focused id); also re-focuses the last remaining execution if the cleared id was focused.
  - `verboseLogging: boolean` and `setVerboseLogging(v)` — persisted in `localStorage` under `resmon.verboseLogging` .
- **Local state in `MonitorPage` :**
  - `routines: RoutineLite[]` — catalogue loaded once from `GET /api/routines` .
  - `execRoutineId: Record<number, number | null>` — per-execution routine id lookup, populated from `GET /api/executions/{id}` for any execution id that is not already

mapped. Manual runs resolve to `null`.

- **Derived values:**
  - `routineNameById` — `Record<number, string>` memoized from the `routines` array.
  - `focusedExec` — `activeExecutions[focusedExecutionId] ?? null`.

Key Interactions and Events

- **Tab rendering.** `executionOrder.map(id => …)` renders one `<div role="tab">` per id. Each tab carries:
  - a status dot whose class is produced by `statusDotClass(status)` — one of `mon-tab-dot-running`, `mon-tab-dot-cancelling`, `mon-tab-dot-completed`, `mon-tab-dot-failed`, `mon-tab-dot-cancelled`;
  - `#<id>` label;
  - the execution-type label from `typeLabel()` (`Deep Dive` / `Deep Sweep` / `Automated Sweep`);
  - a `mon-tab-routine` span with the originating routine name, when `routineNameById[routineId]` resolves;
  - the elapsed clock rendered by `formatClock(exec.elapsedSeconds)` (`H:MM:SS` when an hour has passed, `MM:SS` otherwise);
  - a `×` close button that calls `clearExecution(id)` only when `isTerminal(status)` is `true` (statuses `completed`, `failed`, `cancelled`).
- **Tab activation.** A click, or `Enter` / `Space` keypress, calls `focusExecution(id)`; the active tab is styled with `mon-tab-active` and `aria-selected={true}`.
- **Toolbar controls (visible whenever a tab is focused):**
  - Verbose-logging checkbox wired to `verboseLogging` / `setVerboseLogging`.
  - When the focused execution is terminal: a "View Report" button (`navigate('/results?exec=<id>')`) and a "Clear Page" button (`clearExecution(focusedExec.executionId)`).
- **Detail pane layout.** When `focusedExec` exists: `ExecutionHeader`, `PipelineStages`, a two-column layout with `RepoProgressGrid` and `StatsCounters`, and `LiveActivityLog` at the bottom. `LiveActivityLog` receives `focusedExec.events` and the `verbose` flag.

Error and Empty States

- **Empty state (no active executions).** When `executionOrder.length === 0`, the page renders the `PageHelp` panel and a single `mon-empty` block inviting the user to launch a Dive or Sweep. No tab strip is shown.

- **No tab focused.** When `executionOrder.length > 0` but `focusedExec` is `null`, the tab strip is rendered and the detail region renders a `mon-empty` block reading "Select an execution above to view its live progress."
- **Routine catalogue fetch failure.** If `GET /api/routines` fails, the `routines` array stays empty; tabs for routine-fired executions simply omit the `mon-tab-routine` span.
- **Per-execution detail fetch failure.** If `GET /api/executions/{id}` fails during routine-id enrichment, the entry is recorded as `null` so it is not retried, and the tab is rendered without a routine name.
- **Progress poll failures.** Handled inside `ExecutionContext.spawnPollerFor`: errors are swallowed and the next 1-second tick retries. The Monitor page does not surface a poll-error banner.

Backend

API Endpoints

- `GET /api/executions/active` — returns `{"active_ids": [...]}` from `progress_store.get_active_ids()`. Polled by `ExecutionContext` every 3 seconds to attach background-initiated runs and detect dropouts.
- `GET /api/executions/{exec_id}` — returns the execution row from SQLite; used to enrich `ActiveExecution.routine_id` and to resolve `parameters.repositories` when attaching a newly detected active run.
- `GET /api/executions/{exec_id}/progress/events` — returns the full event list for an execution. Prefers live in-memory events from `progress_store` while the execution is registered; falls back to `get_progress_events(conn, exec_id)` after cleanup. Response headers force `Cache-Control: no-store, no-cache, must-revalidate` to prevent Chromium from serving stale ticks.
- `GET /api/executions/{exec_id}/progress/stream` — SSE endpoint. Emits `event: progress` frames with a JSON body per event and `id: <cursor>` headers; honours the `last_event_id` query parameter. Sends `: heartbeat\n\n` comments every ~300 ms to flush buffers, and on terminal status drains any persisted-but-unread events from the DB (with up to 5 retry sleeps) before closing. The Monitor page itself does not subscribe to this endpoint; it is the designated mechanism for external/alternative clients and matches the cloud progress transport schema (ADQ-15).
- `POST /api/executions/{exec_id}/cancel` — invoked by cancel buttons in `ExecutionHeader` and `FloatingWidget`. Returns `409 Execution not running` if the id is not active; otherwise

calls `progress_store.request_cancel(exec_id)` and returns `{"status": "cancellation_requested"}`.
- `GET /api/routines` — used once on mount to populate `routines` for the routine-name lookup.

Request/Response Patterns

- **Event polling.** `GET /api/executions/{id}/progress/events` returns a JSON array of `ProgressEvent` objects (`{type, timestamp, …}`). `ExecutionContext.spawnPollerFor` maintains a per-id cursor (`eventCursorRef`), slices newly appended events with `allEvents.slice(cursor)` , and folds each event through `updateExecutionState` .
- **SSE framing.** The `progress/stream` endpoint emits frames of the form `id: <n>\nevent: progress\ndata: <json>\n\n` . Heartbeats are sent as `: heartbeat\n\n` comments. Standard SSE response headers are set (`Cache-Control: no-cache` , `Connection: keep-alive` , `X-Accel-Buffering: no`).
- **Active-id contract.** `GET /api/executions/active` returns a tiny `{active_ids: number[]}` payload — intentionally cheap so the 3-second safety-net poll has negligible cost.
- **Cancel contract.** `POST /api/executions/{id}/cancel` is cooperative: it sets a cancel flag in `progress_store` ; the sweep/summarization engine finishes its current batch, flushes partial results, and emits a `cancelled` terminal event before exiting. The frontend optimistically transitions the row's status to `cancelling` before the POST resolves.

Persistence Touchpoints

- `progress_store` (in-memory) owns the live event queue, the active-id set, and the cancel flag. It is registered per execution when the backend starts a run and deregistered after cleanup.
- `progress_events` table (via `save_progress_events` / `get_progress_events` in `implementation_scripts.database`) persists the full event sequence after an execution ends, so the detail pane can still replay history after `progress_store` cleans up.
- `executions` table supplies the metadata rows read by `GET /api/executions/{id}` (status, `execution_type` , `start_time` , `parameters` , `routine_id`).
- `verboseLogging` preference is persisted client-side in `localStorage` only; no backend persistence.

Execution Side Effects

- **Attach-on-mount.** When `ExecutionProvider` mounts, it polls `/api/executions/active` immediately and every 3 s thereafter. Any id not yet tracked triggers `GET /api/executions/{id}` , resolves repositories from `parameters.repositories` (or

`parameters.repository`), and calls `startExecution(id, execution_type, repos, start_time)` — which spawns the per-id 1-second progress poller and 1-second elapsed-time ticker.

- **Terminal-event handling.** The progress poller folds each new event through `updateExecutionState` and detects terminal events (`complete` / `cancelled` / `error`). On the first terminal transition for an id, it:
  i. stops the per-id poller via `stopPollingFor(execId)` ;
  ii. sets `isWidgetPulsing = true` (the FloatingWidget pulse animation);
  iii. calls `broadcastCompletion(execId, {...})` , which increments `completionCounter` and dispatches `window.dispatchEvent(new CustomEvent('resmon:execution-completed', {detail}))` — once per id, guarded by `completionBroadcastRef` ;
  iv. calls `maybeNotifyCompletion(snapshot)` to fire an OS desktop notification subject to `/api/settings/notifications` and, for routine-fired runs, the per-routine `notify_on_complete` flag.
- **Active-dropout safety net.** Any id that was tracked but has dropped out of `/api/executions/active` is treated as completed by `handleActiveDropouts` ; `broadcastCompletion` is idempotent per id, so the terminal-event path and the dropout path coexist without double-firing.
- **FloatingWidget coupling.** The widget reads `activeExecution` (the focused `ActiveExecution`), `executionOrder` , `focusedExecutionId` , and `isWidgetPulsing` from the same context. It auto-minimizes on navigation to `/monitor` , `/results*` , or `/calendar` , renders a stacked-execution strip with an "Also running" popover when `executionOrder.length >= 2` , and uses its own `formatElapsed(seconds)` helper (`MM:SS` only) distinct from the Monitor page's `formatClock(seconds)` (`H:MM:SS` when an hour has elapsed).
- **Event model reducer.** `updateExecutionState` applies the following event types: `repo_start` sets `currentRepo` / `currentRepoIndex` / `totalRepos` and marks the repository as `querying` ; `repo_done` marks `done` and adds to `resultCount` ; `repo_error` marks `error` ; `dedup_stats` recomputes `resultCount = total - invalid` and sets `newCount = new` ; `stage` sets `currentStage` ; `complete` sets terminal status and final counts; `cancelled` sets status to `cancelled` ; `error` sets status to `failed` .

Monitor Page — Info Document

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Backend

API Endpoints

Request/Response Patterns

Persistence Touchpoints

Execution Side Effects